

CPU Zamanlama Algoritmaları

İşletim Sistemleri — Konu 2

1. Zamanlamanın Amacı

CPU zamanlayıcısı (scheduler), ready durumundaki süreçler/thread'ler arasından bir sonraki hangisinin CPU'yu kullanacağına karar verir. İyi bir zamanlama algoritması şu hedefleri dengeler:

- **CPU kullanımı:** CPU'nun boşa kalma süresini azaltmak.
- **Verim (throughput):** Birim zamanda tamamlanan süreç sayısı.
- **Dönüş süresi (turnaround time):** Bir sürecin gelişinden bitişine kadar geçen süre.
- **Bekleme süresi (waiting time):** Sürecin ready kuyruğunda harcadığı toplam süre.
- **Yanıt süresi (response time):** İlk CPU tahsisine kadar geçen süre (etkileşimli sistemlerde kritik).

2. Round-Robin (RR)

Round-robin, her sürece sabit bir zaman dilimi (**quantum**) tahsis eder ve süreçleri dairesel bir kuyrukta sırayla çalıştırır. Quantum dolmadan süreç kendiliğinden bloke olmazsa, zamanlayıcı context switch yapıp süreci kuyruğun sonuna koyar.

Quantum seçiminin etkisi (kritik ödünleşim):

- **Quantum çok küçükse:** context switch sıklığı artar. Context switch saf ek yüküdür (Konu 1) — CPU zamanının önemli bir kısmı kullanıcı kodu yerine yazmaç kaydetme/yükleme ve (süreçler arasıysa) TLB flush'a gider. Aşırı durumda sistem neredeyse tüm zamanını context switch yaparak geçirir ve verim çöker.
- **Quantum çok büyükse:** RR, pratikte First-Come-First-Served'e (FCFS) yaklaşır ve yanıt süresi kötüleşir — etkileşimli bir sistem "donmuş" hissettirebilir.

Pratik kural: quantum, tipik bir sürecin CPU patlaması (CPU burst) süresinden biraz büyük seçilmelidir; genel öneri 10-100 milisaniye aralığıdır.

3. Shortest Job First (SJF)

SJF, ready kuyruğundaki süreçler arasından **tahmini CPU süresi en kısa olanı** seçer. Kanıtlanabilir biçimde ortalama bekleme süresini minimize eder (verilen CPU süreleri

doğruysa).

- **Non-preemptive SJF**: Bir süreç başladıktan sonra, daha kısa bir süreç gelse bile yarıda kesilmez.
- **Preemptive SJF (Shortest Remaining Time First, SRTF)**: Yeni gelen sürecin kalan süresi, çalışmakta olanınkinden kısaysa çalışan süreç kesilir.

Zayıf yönü — açlık (starvation): Sürekli kısa süreçler geliyorsa uzun bir süreç süresiz ertelenebilir. Ayrıca CPU patlama süresi gerçekte önceden bilinmez; genellikle geçmiş patlamaların üstel ortalamasıyla **tahmin edilir**.

4. Öncelikli Zamanlama (Priority Scheduling)

Her sürece bir öncelik değeri atanır; zamanlayıcı en yüksek öncelikli ready süreci seçer. Round-robin ile birleştirilebilir (aynı öncelikteki süreçler RR ile paylaşır).

Açlık sorunu ve çözümü — yaşlandırma (aging): Düşük öncelikli bir süreç yüksek öncelikli süreçler yüzünden hiç çalışamayabilir. Yaşlandırma tekniği, bir sürecin ready kuyruğunda geçirdiği süreyle orantılı olarak önceliğini kademeli artırır; böylece sonsuz bekleyen bir süreç eninde sonunda en yüksek önceliğe ulaşır çalışır.

5. Zamanlama Kuyruğu Veri Yapısı ve Karmaşıklık

Basit bir FCFS/RR kuyruğu bir **queue** (dairesel liste) ile $O(1)$ ekleme/çıkarma sağlar. SJF ve öncelikli zamanlamada ise her seçimde en küçük (kısa süre/yüksek öncelik) elemanı bulmak gerekir:

- Sıralanmamış liste ile: her seçim $O(n)$.
- Bir **min-heap (öncelik kuyruğu)** ile: ekleme ve çıkarma $O(\log n)$; n süreç için tüm zamanlama dizisi $O(n \log n)$ karmaşıklığına sahiptir. Bu yüzden üretim zamanlayıcıları (ör. Linux'un eski $O(1)$ zamanlayıcısı ve sonraki Completely Fair Scheduler'ın kırmızı-siyah ağacı) sıralı veri yapıları kullanır.

6. Karşılaştırma Tablosu

Algoritma	Preemptive?	Açlık riski	Tipik kullanım
FCFS	Hayır	Uzun süreç kısıyı bloklar (convoy etkisi)	Basit toplu (batch) sistemler
Round-Robin	Evet (quantum ile)	Yok	Etkileşimli/paylaşımlı zaman sistemleri
SJF / SRTF	Duruma göre	Var (uzun süreçler)	Ortalama bekleme süresi kritikse
Öncelikli	Duruma göre	Var (aging olmadan)	Gerçek zamanlı / öncelik farkı olan işler

7. Özet

- Zamanlama, CPU kullanımı, verim, dönüş süresi, bekleme süresi ve yanıt süresi arasında ödünleşim yapar.
- Round-robin'de quantum seçimi context switch maliyeti ile yanıt süresi arasında bir ödünleşimdir; çok küçük quantum sistemi context switch'e boğar.
- SJF ortalama bekleme süresini minimize eder ama açlığa açıktır; öncelikli zamanlamada açlık, yaşlandırma (aging) ile çözülür.
- Öncelik kuyruğu tabanlı zamanlayıcıların tipik karmaşıklığı $O(n \log n)$ 'dir.